Atividade Prática - Aula 04

Login com JWT, Pinia e Proteção de Rotas (Trilhas API + Frontend)

Baseado em: roteiro da **Aula 04 — trilha API (Shortz-App API)** e roteiro da **Aula 04 — trilha Frontend (Shortz-App)**

Pré-requisito: Atividades das Aulas 01, 02 e 03 concluídas nas duas trilhas — API com o módulo `modules/user/` já persistindo cadastros de verdade (`POST /register` , `GET /profile/:username`), `middlewares/asyncHandler.js` e `middlewares/errorHandler.js` já registrados; front com a tela de Registro funcionando de ponta a ponta.

Ao final desta atividade, você deve ter, no seu próprio projeto:

Do lado do back-end (API):

- `POST /api/login` devolvendo um **token JWT** válido e os dados do usuário autenticado;
- `GET /api/profile/me` , protegida por um middleware `isAuthenticated` — `401` sem token válido, dados do próprio usuário com token válido;
- `POST /api/logout` existindo, embora não revogue nada no servidor (discussão da Etapa 8).

Do lado do front-end:

- **Pinia** finalmente registrada como plugin da aplicação, com uma **store de autenticação** (`stores/auth.js`) guardando token e dados do usuário, persistidos em `localStorage` ;
- Dois **interceptors** novos no Axios: um de requisição (anexa `Authorization: Bearer <token>` automaticamente) e uma extensão do de resposta (logout automático em qualquer `401`);
- **Bootstrap 5** incluído via CDN pela primeira vez, com uma identidade visual mínima (cor de marca) aplicada à tela de Login;
- A tela de **Login** funcional, um **guarda de rotas** protegendo telas que exigem autenticação, e um **logout** funcional de ponta a ponta.

Você também deve ser capaz de **explicar com suas palavras**:

- O que é um JWT (as três partes, o que é assinado vs. o que é só codificado) e por que o payload nunca deve conter dados sensíveis;
- Por que "logout", num sistema JWT stateless, não revoga nada no servidor — e o que mudaria isso;

- Por que ler o token do `localStorage` diretamente dentro do interceptor do Axios evita um problema de dependência circular entre arquivos.

Recupere a sua ficha de preparação. Hoje ela ganha uma linha nova:

Dado	Seu valor
Cor de marca do seu projeto (hex)	*(escolha uma cor que identifique visualmente o seu projeto — pode ser inspirada no tema: verde para GreenLife, roxo para MusicVibe, etc.)*

PARTE A — Backend: Login, JWT e Middleware de Autenticação

Checklist das tarefas:

- [ ] `jsonwebtoken` instalado
- [ ] `.env` atualizado com `JWT_SECRET` e `JWT_EXPIRES_IN`
- [ ] `config/jwt.js` criado e programado com as funções de carregamento e segurança do `.env`, geração e verificação de tokens
- [ ] `middlewares/auth.js` criado e programado
- [ ] `userService.js` atualizado com `loginUser` e `getUserProfile`
- [ ] `userValidator.js` atualizado com `loginValidator`
- [ ] `userController.js` atualizado com `login`, `logout`, `getMyProfile`
- [ ] `userRoutes.js` atualizado, com `/profile/me` **antes** de `/profile/:username`

Checklist dos testes:

- [ ] Login com sucesso devolve `token` e `user` (incluindo `isAdmin`)
- [ ] Senha errada devolve `500` com mensagem genérica
- [ ] `GET /profile/me` sem token devolve `401`
- [ ] `GET /profile/me` com token válido devolve os dados do usuário
- [ ] `GET /profile/me` com token inválido devolve `401`

PARTE B — Frontend: Login, Pinia e Proteção de Rotas

Checklist das tarefas:

- [ ] `createPinia()` registrado em `main.js`, antes de `.use(router)`

- [ ] `stores/auth.js` criado, com persistência via `localStorage`
- [ ] Interceptor de requisição anexando `Authorization` quando existe token
- [ ] Interceptor de resposta limpando a sessão e redirecionando em qualquer `401`
- [ ] Os nomes de chave usados aqui batem exatamente com os usados na store (Etapa 2)
- [ ] Bootstrap 5 (CSS) incluído via CDN
- [ ] `assets/main.css` criado, com a cor de marca do seu projeto
- [ ] Telas construídas de hoje em diante usam classes Bootstrap; as telas das Aulas 01–03 permanecem como estão, por ora
- [ ] Tela de Login funcional, chamando `authStore.login(...)`
- [ ] Destino padrão pós-login ajustado ao nome real da sua rota principal
- [ ] Guarda de rota bloqueando acesso direto a rotas com `requiresAuth: true`
- [ ] `useAuthStore()` chamado **dentro** do callback, não no topo do arquivo
- [ ] Navbar mostra links diferentes conforme o estado de login
- [ ] Logout limpa a sessão e redireciona ao Login

Checlist de testes

- [ ] Login funcional → redireciona à tela principal do seu projeto.
- [ ] F5 na página logado → sessão persiste (confira `localStorage` no DevTools).
- [ ] Logout → depois disso, digitar a URL de uma rota protegida na barra de endereço deve redirecionar ao Login, com `?redirect=...` na URL.
- [ ] Login a partir dessa tela redirecionada → deve voltar exatamente para a rota original.
- [ ] Com a sessão ativa, edite manualmente o token no `localStorage` (DevTools) para um valor qualquer, recarregue numa rota protegida → deve ser automaticamente redirecionado ao Login (prova do interceptor de `401`).

Entrega

Envie em resposta a esta tarefa:

1. Na subpasta `01 - Docs` do seu projeto frontend, dentro de uma nova subpasta `atividade04`:
   - A ficha em markdown, atualizada com a cor de marca;
   - Um print de cada `curl` do teste da Parte A, em `.jpg`;
   - Um print da tela de Login com o Bootstrap/cor de marca aplicados, em `.jpg`, com o nome `login-estilizado.jpg`;
   - Um print mostrando o redirecionamento ao Login ao tentar acessar uma rota protegida deslogado, com a URL exibindo `?redirect=...`, em `.jpg`;
   - Um print do Console mostrando `Perfil autenticado OK:`, em `.jpg`;

    - Um print demonstrando o logout automático após editar o token manualmente, em `.jpg`;
    - Atualize `checklists.md` com todos os checklists desta atividade marcados.
2. Um arquivo `.zip` do seu projeto de back-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-backend.zip`;
3. Um arquivo `.zip` do seu projeto de front-end (sem `node_modules`), nomeado `<NomeSobrenome>-<NomeProjeto>-frontend.zip`, agora incluindo `stores/`.